Software Requirements Specification

for

EventBridge AI

Version 1.0

Prepared by Group #49

Sir Syed University of Engineering & Technology

Date: 30 March,2026

Team

Member Name	Primary Responsibility
Laiba Rafique (2022F-BSE-064)	Development (AI, Frontend, Backend)
Ahmer Zubair (2022F-BSE-165)	UI, UX, Graphic Designing, Research and Documentation
Muhammad Hashir (2022F-BSE-289)	SQA, Research and Documentation

Table of Contents

1. Introduction

1.1 Purpose

EventBridge AI is designed to address a specific and recurring challenge faced by individuals and communities in cities like Karachi: fragmented and disorganized event discovery and management. These communities, like IEEE, GDSC, ACM, and university clubs, frequently organize events for learning, networking, skill development, and collaboration. However, due to the lack of a centralized digital platform, such events are scattered across informal channels such as WhatsApp groups, Instagram Stories, and isolated websites. This fragmentation leads to three key problems: poor discoverability, low turnout, and missed opportunities for Individuals and Communities alike.

The proposed platform, EventBridge AI, is a centralized, AI-powered ecosystem that helps individuals discover, join, and interact with events in meaningful ways. It is not just a listing site; it intelligently understands user interests and improves participation through timely, relevant suggestions. Communities benefit from structured tools for event publishing, community building, and feedback analysis.

The broader purpose of this project includes:

- Eliminating the disconnection between events and their relevant audiences by creating a smart central hub for all Individuals and communities.
- Using AI-powered personalization and filtering to tailor event recommendations to individual preferences and community interests.
- Offering semantic search capabilities to allow users to find events not just by keywords but by context (e.g., "free tech workshops this weekend").
- Supporting future features like event check-in, attendance tracking, and feedback dashboards to help communities grow strategically.

Overall, the platform enhances engagement, reach, and operational efficiency for both event participants and organizers.

1.2 Scope

EventBridge AI is envisioned as a web-based application built with a scalable backend (Laravel PHP), a frontend interface (HTML/CSS/JS), and an intelligent AI engine (Python + Django). The system is designed to support seamless interaction between individuals and communities, focusing on discoverability and personalization.

The scope of the project includes:

- **User Registration and Profiles:** Individuals can register and manage their profiles by defining personal interests. The system enables users to follow communities.
- **Community Registration:** Organizations or clubs must register by providing official entity details through a specialized onboarding form.
- **Administrative Approval:** To ensure platform quality, Community accounts remain inactive until the System Admin reviews and approves the registration.

- **Secure Authentication:** Upon approval, the community can log in using their credentials to access their community dashboard.
- **Event Management System:** End-to-end management of event listings, including title, poster, type (online/offline), fee (paid/free), and tags.
- **Explore Feed:** Users can discover communities to follow and view event details, including options to register or provide feedback. The system displays events from followed communities, personalized recommendations, and overall new live events available to all users.
- **Semantic Search:** A NLP-driven search engine that allows users to search using natural language queries for more relevant results.
- **Recommendation Engine:** The system suggests events based on user activity, preferences, and previous attendance. These recommendations are generated using Collaborative Filtering or Content-Based Filtering, or a Hybrid Recommendation approach.
- **Notification System:** The system provides a centralized notification module for both Individuals and Communities, alerting them of key activities and relevant actions.
- **Administrative Controls:** The platform administrator oversees system operations and manages community registrations. Their primary responsibility is to review, approve, or reject community registration.
- **Cross-Community Features:** Support for inter-community collaborations, shared events, and city-wide visibility of educational and professional initiatives.

Future expansion includes attendance tracking, sentiment analysis on feedback, and dashboards with visual insights for organizers to assess engagement levels.

This platform aligns with the following Sustainable Development Goals (SDGs):

- **Quality Education (SDG 4):** Enhancing learning opportunities through accessible events.
- **Industry, Innovation, and Infrastructure (SDG 9):** Providing an intelligent, scalable infrastructure for digital communities.
- **Partnerships for the Goals (SDG 17):** Promoting inter-community collaboration for shared learning and growth.

1.3 Definitions

This section defines key technical and domain-specific concepts used within the Event Bridge AI system to ensure clarity and consistency across stakeholders.

- **Artificial Intelligence (AI):** The simulation of human intelligence in computer systems, enabling tasks such as learning, reasoning, pattern recognition, and decision-making. In this system, AI is primarily used for semantic search and personalized event recommendations.

- **Large Language Model (LLM):** A deep learning model trained on large-scale textual data, capable of understanding and generating human-like language. It may be utilized for advanced query understanding in the platform.
- **Natural Language Processing (NLP):** A branch of AI that enables machines to understand, interpret, and process human language. It is used to interpret user search queries in semantic search.
- **Semantic Search:** An AI-driven search technique that interprets user intent and contextual meaning rather than relying solely on keyword matching, enabling more accurate event discovery.
- **Recommendation Engine:** A system that analyzes user behavior, interests, and interaction history to suggest relevant events using similarity scoring and ranking algorithms.
- **Vector Embedding:** A numerical representation of textual or behavioral data in a high-dimensional space, enabling similarity comparison between users and events.
- **Vector Database:** A specialized database optimized for storing and querying vector embeddings, used for efficient similarity search in AI-powered features.
- **Event Lifecycle:** The end-to-end process of managing an event, including creation, editing, publishing, registration, attendance tracking, and feedback collection.
- **Community Collaboration:** A system feature that allows multiple communities to co-manage and organize events with shared ownership and responsibilities.
- **Personalized Events:** A dynamically generated list of recommended events tailored to an individual user based on preferences, behavior, and AI-driven insights.

1.4 Acronyms and Abbreviations

This section lists all abbreviations and acronyms used throughout this document for ease of reference.

- **AI** – Artificial Intelligence
- **API** – Application Programming Interface
- **DB** – Database
- **FYP** – Final Year Project
- **FR** – Functional Requirements
- **LLM** – Large Language Model
- **NLP** – Natural Language Processing
- **NFR** – Non-Functional Requirements
- **SDG** – Sustainable Development Goals
- **UI** – User Interface
- **UX** – User Experience
- **Vector DB** – Vector Database

2. Project Planning and Management

Project planning and management are foundational pillars in the development lifecycle of EventBridge AI. Given the academic constraints and technical complexity of the system, it is essential to adopt structured, well-documented, and phased planning strategies to ensure timely progress and quality deliverables. This section outlines the strategic thinking, scheduling, work segmentation, and risk-mitigation efforts applied to the development of EventBridge AI.

2.1 SWOT Analysis

For the EventBridge AI project, the SWOT analysis informs:

- The technology stack selection (leveraging strengths in AI expertise).
- Design decisions that avoid complexity pitfalls.
- Features that align with user needs and market gaps.
- Safeguards against competitive and regulatory threats.

The following presents a comprehensive SWOT analysis specifically tailored to the strategic, technical, and operational dimensions of the EventBridge AI platform.

Strengths:

- **AI-Driven Personalization:** EventBridge AI utilizes vector-based semantic search and user interest to tailor event recommendations to each user, increasing relevance and engagement.
- **Centralized Community Ecosystem:** Unlike fragmented event platforms, it brings together all city-wide communities into a single, structured digital hub.
- **Scalable Modular Architecture:** The platform is built with a layered, service-oriented architecture that supports expansion across cities or institutions.

Weaknesses:

- **Initial Data Scarcity:** The recommendation engine requires historical user interaction data, which may not be available during early usage.
- **Technical Complexity:** Combining PHP, Python, LLMs, and vector databases adds layers of integration complexity, increasing the need for testing and coordination.
- **Non-technical User Adjustment:** First-time users may need onboarding to understand filters, semantic search, and AI-based personalization.

Opportunities:

- **Post-COVID Digital Shift:** The rise of online/hybrid events creates a sustained demand for centralized digital engagement tools.
- **Academic Adoption:** Universities and student bodies can adopt the system as their official platform for event management and inter-community networking.

- **Market Differentiation:** Eventbridge AI distinguishes itself by providing an ecosystem for academic and professional Communities to centralize their activities. Unlike generic event tools, it focuses on structured community management and intelligent discovery to foster long-term engagement and seamless networking within a unified digital space.
- **Future Expansion:** Potential to commercialize the platform for educational boards, youth conferences, or city-level event organizers.

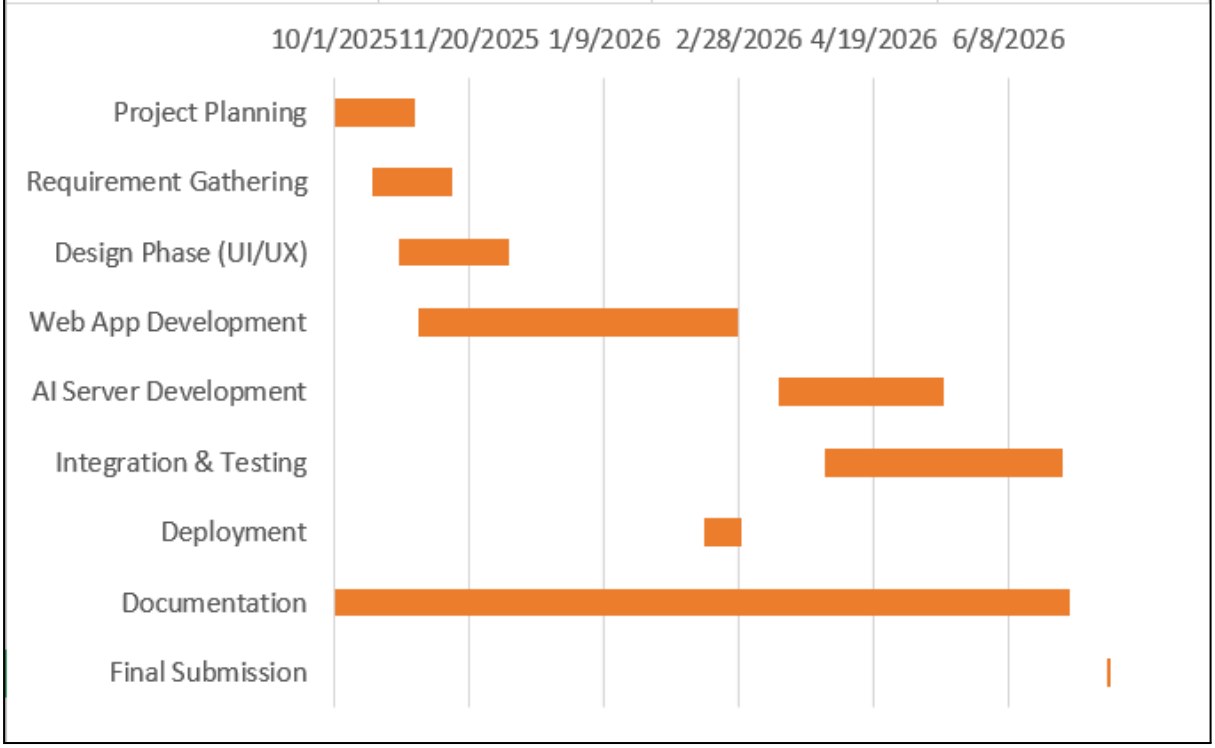
Threats:

- **Regulatory Constraints:** Data privacy laws must be strictly followed due to the involvement of user interaction history and behavioral profiling.
- **Technology Evolution:** Advancements in vector search engines, LLMs, or better-performing AI APIs could necessitate regular refactoring.
- **Indirect Competitors:** Generic social platforms might start offering similar features, diminishing the platform's uniqueness if not updated regularly.
- **Cybersecurity Risks:** As the system collects user profiles and behavioral data, it becomes a target for potential breaches if not well-secured.

By clearly identifying and addressing these elements, the SWOT analysis empowers the project team to take proactive, informed decisions and adapt to changing environments throughout development and deployment.

2.2 Gantt Chart

Task	Start Date	End Date	Duration (Days)
Project Planning	10/1/2025	10/31/2025	30
Requirement Gathering	10/15/2025	11/14/2025	30
Design Phase (UI/UX)	10/25/2025	12/5/2025	41
Web App Development	11/1/2025	2/28/2026	119
AI Server Development	3/15/2026	5/15/2026	61
Integration & Testing	4/1/2026	6/28/2026	88
Deployment	2/15/2026	3/1/2026	14
Documentation	10/1/2025	7/1/2026	273
Final Submission	7/15/2026	7/16/2026	1



1. Project Planning (Start Date: 10/1/2025, End Date: 10/31/2025, Duration: 30 Days)

- **Description:** The Project Planning phase is the foundational step for setting the direction and scope of the EventBridge AI project. This phase focuses on gathering requirements, defining the project scope, and establishing a high-level architecture.
- **Key Activities:**
    - **Requirement gathering:** Collecting initial system requirements, both functional and non-functional, and aligning them with stakeholders' needs.
    - **Scope definition:** Setting clear objectives for what the project will achieve and what it won't.
    - **Initial architectural planning:** High-level discussions and decisions on the tech stack, tools, and methodologies to be used.

- **Analysis:** This phase has a defined start and end date and occupies 30 days. It is essential because it sets the stage for all future tasks by ensuring the team has a clear understanding of the project requirements, timelines, and available resources.

2. Requirement Gathering (Start Date: 10/15/2025, End Date: 11/14/2025, Duration: 30 Days)

- **Description:** The Requirement Gathering phase builds on the initial project planning by refining and finalizing the system's requirements, including detailed user stories, technical specifications, and acceptance criteria.
- **Key Activities:**
    - **Stakeholder Interviews:** Engaging with end-users, system administrators, and business stakeholders to refine system requirements.
    - **Use Case Development:** Formalizing how different actors (Users, Communities and admin) will interact with the platform.
    - **Validation:** Cross-referencing the gathered requirements with feasibility studies to ensure that the requirements are implementable within the set constraints.
- **Analysis:** This phase is 30 days long, running concurrently with Project Planning for 15 days. This ensures an overlap, allowing for feedback and refinement. It is crucial to gather comprehensive data to inform the design and implementation stages.

3. Design Phase (UI/UX) (Start Date: 10/25/2025, End Date: 12/5/2025, Duration: 41 Days)

- **Description:** The Design Phase is focused on developing the system's UI/UX design. The goal is to create wireframes, user flows, and design prototypes that clearly represent the user experience.
- **Key Activities:**
    - **Wireframing:** Sketching the basic layout and functionality of the platform's user interface.
    - **Prototype Design:** Designing interactive UI prototypes that simulate user interaction with the platform.
    - **User Feedback:** Conducting usability testing with stakeholders to validate design choices before development begins.
- **Analysis:** Lasting 41 days, this phase overlaps slightly with the Requirement Gathering phase, which helps streamline the transition from planning to actual system design. The longer duration reflects the criticality of getting the design right before development begins, as it will directly affect the user experience and system performance.

4. Web Application Development (Start Date: 11/1/2025, End Date: 2/28/2026, Duration: 117 Days)

- **Description:** The Web App Development phase is the core development phase for building the frontend and backend of the platform. This task involves implementing the system's design, developing features, and integrating components.
- **Key Activities:**
    - **Frontend Development:** Coding the user interface (UI) based on the wireframes and prototypes.
    - **Backend Development:** Developing server-side logic, APIs, and database structures using frameworks such as Laravel (PHP) and MySQL.
    - **Integration:** Ensuring the frontend and backend communicate effectively through APIs and data exchange mechanisms.
- **Analysis:** This phase spans 117 days and is the longest single task. The duration is justified given the complexity of developing a fully functional web application with multiple integrations. It is a foundational task that will require ongoing testing and refinement.

5. AI Server Development (Start Date: 3/15/2026, End Date: 5/15/2026, Duration: 92 Days)

- **Description:** This phase focuses on the development of the AI recommendation engine and semantic search functionality. This system will power the personalized event recommendations and search features for the platform.
- **Key Activities:**
    - **AI Model Development:** The system utilizes Generative AI and vector embeddings to power its recommendation engine. It employs content based or collaborative, or hybrid filtering approaches to provide users with relevant event suggestions.
    - **Semantic Search:** Building the ability for users to search for events using natural language queries.
- **Analysis:** The 92-day duration reflects the complexity of Web App Development, as both systems must integrate seamlessly. The AI engine must be functional and well-integrated with the platform before proceeding to the next phase.

6. Integration & Testing (Start Date: 4/1/2026, End Date: 6/28/2026, Duration: 90 Days)

- **Description:** The Integration & Testing phase ensures that all components, including the frontend, backend, and AI server, work together as expected.
- **Key Activities:**
    - **System Integration:** Integrating all platform components (web app, AI model, databases) and ensuring smooth communication between them.
    - **Unit Testing:** Conducting individual unit tests for each module.
    - **User Acceptance Testing (UAT):** Verifying that the system meets user expectations and functional requirements.

- **Analysis:** With a duration of 90 days, this phase is designed to identify and address bugs, integration issues, and performance bottlenecks before deployment. The relatively short duration ensures that the platform will be ready for deployment without unnecessary delays.

7. Deployment (Start Date: 2/15/2026, End Date: 3/1/2026, Duration: 14 Days)

- **Description:** The Deployment phase involves moving the platform from the development environment to a live production environment.
- **Key Activities:**
    - **Cloud Hosting Setup:** Deploying the platform to a cloud provider, ensuring it is accessible to users.
    - **Security Measures:** Implementing encryption, access controls, and security auditing.
    - **Final System Validation:** Ensuring everything works in the live environment as expected.
- **Analysis:** This phase is relatively short, lasting 14 days, as the infrastructure is already in place. The goal is to ensure that the system is properly configured, secure, and fully functional in a production environment.

8. Documentation (Start Date: 10/1/2025, End Date: 7/1/2026, Duration: 30 Days)

- **Description:** The Documentation phase includes writing the technical and user-facing documentation necessary for the platform.
- **Key Activities:**
    - **SRS (Software Requirements Specification):** A formal document that describes the system's purpose, functional and non-functional requirements, and any constraints the software must follow. It serves as a foundation for the entire project development.
    - **SDD (Software Design Document):** A technical document that provides a detailed description of the system's architecture, data structures, and interface design, acting as a primary reference for the implementation phase.
    - **Technical Report:** Academic review document covering progress and learnings.
- **Analysis:** 273 days are allocated for this phase to ensure comprehensive documentation is available for future developers, users, and administrators.

9. Final Submission (Start Date: 7/15/2026, End Date: 7/16/2026, Duration: 1 Days)

- **Description:** The final submission marks the completion of the project. This phase involves wrapping up the project, ensuring everything is documented, and submitting the final deliverables.
- **Key Activities:**

    - **Project Wrap-Up:** Final review of all deliverables, ensuring they meet requirements.
    - **Submission:** The platform is submitted for academic evaluation.
- **Analysis:** This phase lasts 1 day, as it is a final administrative task that marks the official completion of the project.

Buffer Time and Dependencies:

- **Buffer Time:** There is no explicit buffer time shown in this chart, but the overlapping tasks (such as AI Server and Web App Development) provide inherent flexibility. Any delays in one phase could potentially affect the subsequent phases, but the overall project timeline allows for concurrent workstreams to mitigate such risks.
- **Dependencies:**
    - Web App Development is dependent on the Requirement Gathering and Design Phase.
    - AI Server is dependent on the Design Phase and overlaps with Web App Development.
    - Integration & Testing is dependent on the completion of AI Server and Web App Development, ensuring all components work together.

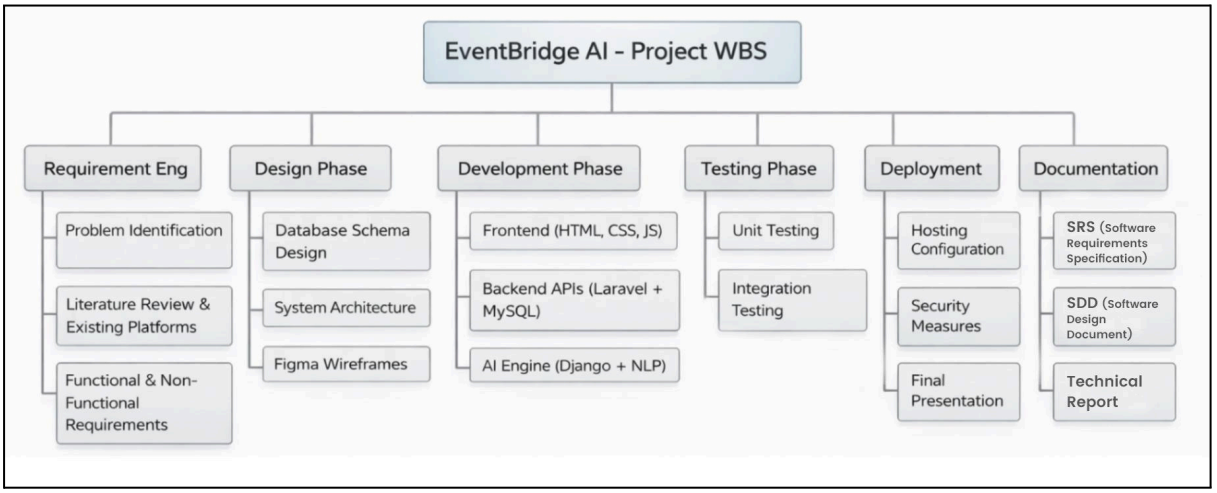
2.3 Work Breakdown Structure (WBS)

In EventBridge AI, the WBS is categorized into six high-level phases, each of which is broken down into clearly defined subtasks:

- **Requirement Engineering:** This initial phase includes identifying the core problem, analyzing existing solutions, and documenting functional and non-functional requirements. It forms the foundation upon which all other development is based.
    - **Problem Identification:** Recognizing and validating the issue of disconnected event ecosystems.
    - **Literature Review & Existing Platforms:** Studying existing solutions like Eventbrite or Meetup to identify feature gaps.
    - **Functional & Non-Functional Requirements:** Creating a structured list of system capabilities and performance expectations.
- **Design Phase:** This includes the visual and technical blueprint of the system.
    - **Database Schema Design:** Mapping out tables, relationships, and data flow.
    - **System Architecture:** Designing the interaction between Laravel backend, Django AI server, and databases.
    - **Figma Wireframes:** Crafting mockups to define frontend user interfaces and user experience flows.
- **Development Phase:** The coding and implementation stage divided by stack layers.

    - **Frontend (HTML, CSS, JS):** Building user-facing components for dashboards, event listings, and profiles.
    - **Backend APIs (Laravel + MySQL):** Creating logic for CRUD operations, authentication, and notifications.
    - **AI Engine (Django + NLP):** Developing the semantic search, event recommendation engine, and embedding-based vector filtering.
- **Testing Phase:** Ensures system quality and reliability through multi-stage evaluations.
    - **Unit Testing:** Verifying individual modules for correctness.
    - **Integration Testing:** Validating how modules interact, especially across frameworks.
- **Deployment:** Preparing the system for live academic use.
    - **Hosting Configuration:** Deploying backend and AI components on cloud platforms.
    - **Security Measures:** Implementing encryption, data validation, and access control.
    - **Final Presentation:** Preparing for academic evaluation, showcasing core deliverables.
- **Documentation:** All reporting and user-assistance material.
    - **SRS (Software Requirements Specification):** A formal document that describes the system's purpose, functional and non-functional requirements, and any constraints the software must follow. It serves as a foundation for the entire project development.
    - **SDD (Software Design Document):** A technical document that provides a detailed description of the system's architecture, data structures, and interface design, acting as a primary reference for the implementation phase.
    - **Technical Report:** Academic review document covering progress and learnings.

Work Breakdown Structure Diagram:



Each box in the WBS diagram represents a work package, a task or collection of closely related subtasks that result in a deliverable. Arrows and branches show the hierarchy, beginning with the overall project at the top and splitting into progressively granular components.

This structured visualization aids in:

- Defining the full scope of work clearly.
- Avoiding task duplication or omission.
- Assigning responsibilities at the subtask level.
- Monitoring deliverables for each development phase.

3. Overall Description

This section presents a comprehensive overview of the EventBridge AI platform, detailing its placement within the software ecosystem, its expected functionality, and the contextual conditions under which it will operate. This provides clarity for both technical evaluators and non-technical stakeholders by illustrating how the system is structured, what it is designed to do, and how users and system components interact.

3.1 Product Perspective

EventBridge AI is positioned as a standalone yet extensible web-based application designed to address the inefficiencies in event discovery and engagement across academic and community ecosystems. Rather than functioning in isolation, EventBridge AI serves as a comprehensive middleware that bridges the gap between fragmented community posts and active event participants.

At its core, EventBridge AI is a multi-layered, modular system composed of the following integrated layers:

- **Frontend Presentation Layer:** A user-facing interface developed using HTML, CSS, and JavaScript, providing dashboards, search modules, and community views.
- **Backend Application Layer:** Built on Laravel (PHP), this layer manages server-side logic, including authentication, event CRUD operations, user profiles, community workflows, and notification routing.
- **AI Services Layer:** Implemented using Django (Python), this component performs semantic search, vector filtering, recommendations, and relevance scoring using embedding models and LLMs.
- **Data Management Layer:** Utilizes MySQL for structured data such as events and user records, and Qdrant (or an equivalent vector database) for managing high-dimensional AI embeddings.

The system follows a decoupled architecture to ensure ease of maintenance, parallel development, and flexibility in updates. JSON-based Rest APIs mediate communication between the Laravel backend and the Django AI server, enabling independent scalability and modular feature enhancements.

3.2 Product Functions

Event Bridge AI provides a comprehensive set of functionalities designed to facilitate event management, discovery, and interaction between users and communities. The system integrates traditional event management features with AI-powered capabilities to enhance user experience and engagement.

The major functional components of the system are described below:

User Management:

The system allows individual users to register, authenticate, and manage their profiles.

- **Account Creation & Authentication:** Users can create accounts and securely log in to the platform.
- **Onboarding & Preferences:** Users provide interests and preferences during onboarding.
- **Personalization Data Usage:** User data is utilized for personalization and recommendation features.

Community Management:

Communities represent organizations that host and manage events.

- **Community Registration:** Communities can register by submitting organizational details.
- **Admin Approval:** Community accounts require administrative approval before activation.
- **Profile Management:** Approved communities can manage their public profiles and information.
- **Team Management:** Communities can maintain and update team member information.

Event Management:

Communities are responsible for managing the complete lifecycle of events.

- **Event Creation & Publishing:** Communities can create, edit, and publish events.
- **Event Details Management:** Communities can manage event schedules, venues, and descriptions.
- **Registration Handling:** Communities can manage user registrations and attendance.
- **Feedback Management:** Communities can collect and manage post-event feedback.

Event Discovery and Interaction:

The platform provides a centralized exploration hub for both users and communities.

- **Event Browsing & Search:** Users can browse and search for events.
- **Event Participation:** Users can register for and attend events.
- **Community Exploration:** Communities can explore events for collaboration opportunities.
- **Follow System:** Users can follow communities to receive updates.

Community Collaboration:

The system enables collaboration between multiple communities.

- **Collaboration Requests:** Communities can send and receive collaboration requests.
- **Joint Event Management:** Multiple communities can co-manage events.
- **Shared Ownership:** Events can have shared ownership between collaborating communities.

AI-Powered Features:

The platform incorporates Artificial Intelligence to enhance discovery and personalization.

- **Semantic Search:** Enables natural language-based event discovery.
- **Recommendation Engine:** Provides personalized event suggestions to users.
- **Behavioral Analysis:** AI models utilize user behavior, preferences, and event data.

Notification System:

The system provides real-time notifications to keep users and communities informed.

- **Event-Triggered Notifications:** Notifications are generated based on system activities and user actions.
- **Alert Types:** Includes registrations, approvals, collaborations, and updates.
- **In-App Delivery:** Notifications are delivered within the platform interface.

Feedback and Interaction System:

The platform allows users to provide feedback on events.

- **User Feedback Submission:** Users can submit textual feedback on events.
- **Visibility Control:** Communities can set feedback visibility as public or private.
- **Manual Moderation:** Feedback is managed manually by the hosting community.

Administrative Control:

The Admin oversees platform operations and governance.

- **Community Approval:** Admin can approve or reject community registrations.
- **System Monitoring:** Admin monitors overall platform activity.
- **Platform Governance:** Admin ensures proper functioning of the system.

3.3 Operating Environment

EventBridge AI is engineered to operate in a cloud-first, browser accessible environment while adhering to modern security and privacy standards. Its infrastructure is optimized for academic institutions but is also capable of scaling to accommodate larger municipal or commercial deployments.

Operating Environment Details:

- **Client-Side Access:** All features are available through major browsers (Chrome, Firefox, Safari, Edge).
- **Server Environment:**
    - Laravel (PHP) hosted on Linux-based virtual servers (NameCheap VPS or equivalent).
    - Django (Python) services are deployed using NGINX + Gunicorn.
    - MySQL for relational data and Qdrant (or similar) for high-dimensional vector embeddings.
- **External Services:**
    - OpenAI or HuggingFace APIs for embeddings and LLM-based tasks.

Constraints and Considerations:

- **Academic Timelines:** The fixed development window demands time-boxed sprints and buffer-inclusive planning.
- **System Dependencies:** Relies on stable internet connections and external APIs which may evolve or change.
- **Resource Optimization:** Embedding computations and semantic ranking require GPU or high-performance server resources, especially during peak usage.
- **Maintenance Separation:** The use of two separate backend technologies requires independent monitoring and deployment cycles.

These environmental definitions and constraints directly shape the development scope, risk planning, and deployment strategy.

3.4 Design and Implementation Constraints

This section describes the limitations and constraints that affect the design and implementation of the Event Bridge AI system.

- **Technology Stack Constraint:** The system is developed using Laravel (PHP) for the main backend and a Python-based framework, Django, for AI services. This requires maintaining interoperability between different technologies.
- **External API Dependency:** The system relies on third-party services such as OpenAI for embedding generation and natural language processing. Changes in API availability, pricing, or performance may impact system functionality.

- **Infrastructure Limitations:** Deployment is expected on virtual private servers (VPS) or cloud-based environments with limited computational resources. AI-related operations such as embedding generation and similarity search may require high-performance resources.
- **Academic Timeline Constraint:** As a Final Year Project (FYP), development must be completed within a fixed academic schedule, limiting the scope for extensive scalability or optimization.
- **Deployment Complexity:** The use of multiple backend services (Laravel + Python AI services) requires separate deployment, monitoring, and maintenance processes.
- **Security and Privacy Requirements:** The system must adhere to standard web security practices, including secure authentication, data protection, and safe API communication.

3.5 Assumptions and Dependencies

This section outlines the assumptions made during system design and the external dependencies required for proper system operation.

Assumptions:

- **Internet Availability:** Users are expected to have access to a stable internet connection to interact with the platform.
- **Valid User Data:** Users and communities provide accurate and valid information during registration and usage.
- **Community Compliance:** Communities follow platform guidelines while creating and managing events.
- **Moderate Initial Scale:** The system is assumed to operate at a moderate scale during initial deployment (academic or limited public usage).

Dependencies:

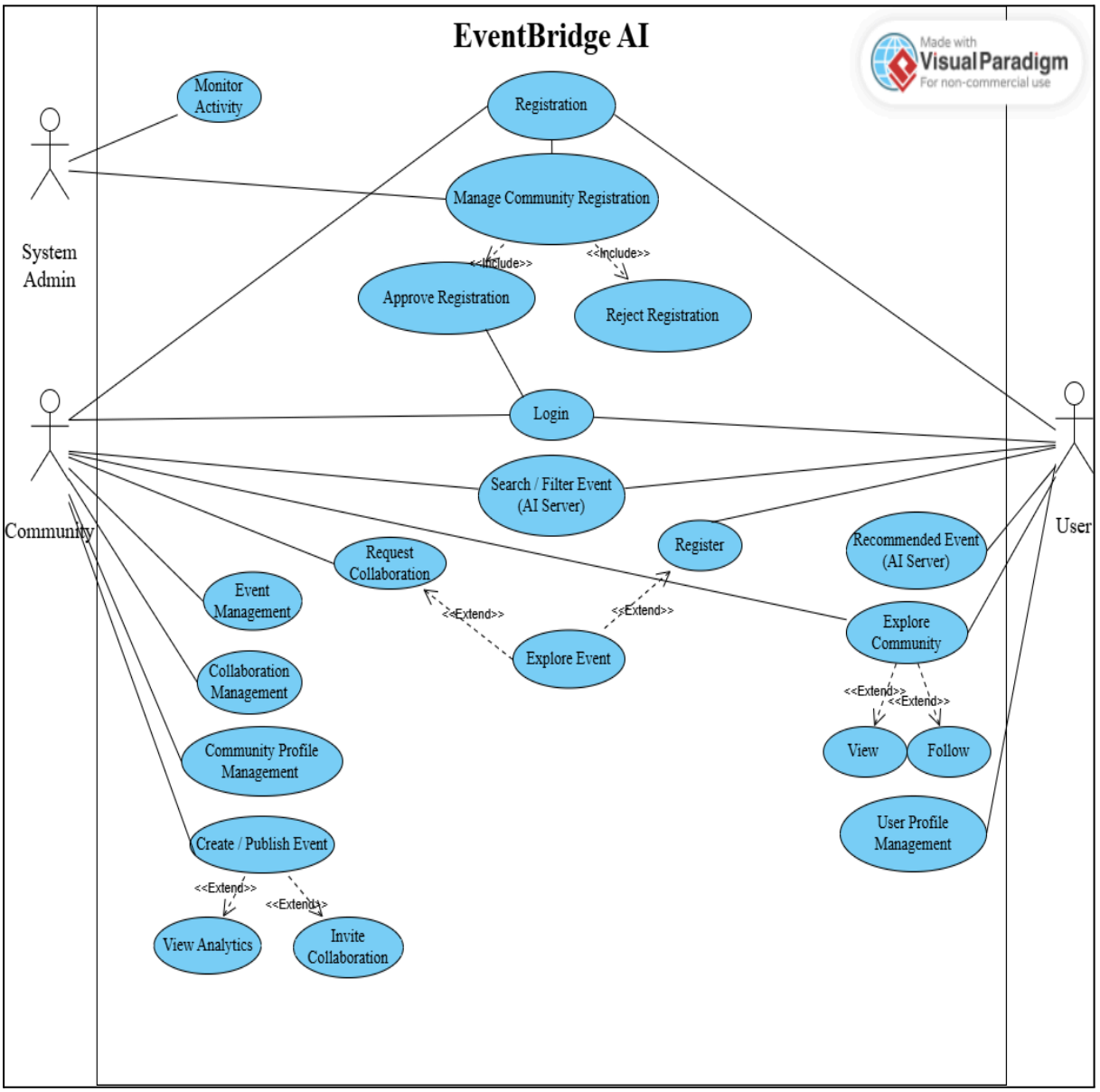
- **Web Browsers:** The system depends on modern web browsers for client-side access.
- **Backend Frameworks:** Laravel (PHP) and Python-based frameworks for core functionality.
- **Database Systems:** MySQL for structured data and Qdrant (or similar) for vector data storage.
- **External AI Services:** APIs such as OpenAI for embeddings and NLP processing.
- **Hosting Environment:** Linux-based servers or cloud infrastructure for deployment.

4. Requirement Identifying Technique

Identifying, refining, and validating system requirements is one of the most critical phases of software development. For EventBridge AI, which is rooted in solving real-world problems through intelligent automation and semantic technologies, the process of requirement gathering was intentionally thorough and iterative.

This section explains the methodologies employed to collect actionable requirements and illustrates how user expectations, system capabilities, and technical feasibility were aligned into one coherent framework. It also introduces system use cases derived from these activities that support the modeling of interaction diagrams.

4.1 Use Case Diagram



4.2 Use Case Description

Use Case Diagram for EventBridge AI illustrates the key actors involved in the system (Individuals, Communities, and Admin) and their respective interactions with the system's functionality. Each use case represents a functional requirement or interaction, showing how users engage with the platform and how their activities are processed by the system.

Actors in the System:

1. **Individual User:** The User is an Individual or attendee who engages with the platform to discover events and receive recommendations.
2. **Community:** The Community is responsible for creating and managing events within EventBridge AI.
3. **Admin (System Admin):** The Admin is a system-level actor responsible for accepting and rejecting the request for community account creation.

Key Use Cases and Interactions:

Individual User:

1. **Register (Individual User):**
    - **Description:** This use case allows an individual to create a new account on the EventBridge AI platform to access user-level features.
    - **Interaction:** The user submits their details (email, password, etc.), which the system stores to create their profile and manage access.
2. **Login (Individual User):**
    - **Description:** This functionality enables registered users to authenticate and securely access their personalized dashboard.
    - **Interaction:** The user enters their credentials, and the system verifies them against the database to grant access.
3. **User Profile Management (Individual User):**
    - **Description:** This allows users to update and manage their personal details, preferences, and account settings.
    - **Interaction:** The user edits their profile information, and the system saves and updates these changes in the database.
4. **Search / Filter Event (AI Server) (Individual User):**
    - **Description:** This allows users to search for events using keywords, filters, or natural language queries. The platform uses AI-driven search to provide relevant results.
    - **Interaction:** The user performs a search, and the AI server queries the event database to return tailored results based on the search criteria.

5. **Recommended Event (AI Server) (Individual User):**
    - **Description:** The system provides personalized event suggestions to the user based on their preferences and past behavior.
    - **Interaction:** The AI Recommendation Engine analyzes user data and displays customized event suggestions exclusively on the user's interface.
6. **Explore Community (Individual User):**
    - **Description:** This allows users to browse through different registered communities on the platform to find groups that match their interests.
    - **Interaction:** The user browses community profiles. They can choose to "View" specific community details or "Follow" a community to receive future updates.
7. **Explore Event (Individual User):**
    - **Description:** Users can browse through various active event listings on the platform.
    - **Interaction:** The user explores available events. While viewing a specific event, they can choose to "Register" to officially attend it.

Community:

1. **Registration (Community):**
    - **Description:** This use case allows an organization to submit a request to create a community account on the platform.
    - **Interaction:** The community submits their organizational details, which are sent to the System Admin for approval before the account is fully activated.
2. **Login (Community):**
    - **Description:** This enables approved communities to authenticate and access their organizational tools and features.
    - **Interaction:** The community enters their credentials, and the system verifies them to grant access to the community dashboard.
3. **Community Profile Management (Community):**
    - **Description:** This allows community administrators to update their public profile, description, and organizational settings.
    - **Interaction:** The community edits their public-facing details, and the system updates their profile across the platform.
4. **Event Management (Community):**
    - **Description:** This provides a centralized hub for communities to oversee their overall portfolio of past, present, and drafted events.

- **Interaction:** The community navigates their dashboard to track, edit, or manage the lifecycle of all their associated events.

5. **Create / Publish Event (Community):**
    - **Description:** Communities can generate new events by providing details such as event name, category, location, time, and description.
    - **Interaction:** The community fills in the details and publishes the event. From here, they can also "View Analytics" for that specific event or "Invite Collaboration" from other communities.
6. **Explore Event (Community):**
    - **Description:** Communities can browse the platform to see events hosted by other organizations for networking or market research.
    - **Interaction:** The community browses event listings. If they find an aligned event, they can "Request Collaboration" to co-host or sponsor.
7. **Search / Filter Event (AI Server) (Community):**
    - **Description:** This functionality allows communities to search for events based on keywords, filters, or natural language queries to find partnership opportunities.
    - **Interaction:** The community inputs a search query, and the AI server processes it to return relevant event results.
8. **Collaboration Management (Community):**
    - **Description:** This enables communities to manage incoming and outgoing collaboration requests to share resources and co-host programs.
    - **Interaction:** The community uses the system to accept, reject, or track partnership requests with other organizations.

Admin:

1. **Monitor Activity (System Admin):**
    - **Description:** Admins can oversee general platform operations, user behavior, and overall system health to ensure a safe environment.
    - **Interaction:** The admin views system dashboards and activity logs to track platform usage and flag any issues.
2. **Manage Community Registration (System Admin):**
    - **Description:** Admins are responsible for reviewing incoming account creation requests from new communities.
    - **Interaction:** The admin reviews the community's submitted details and will either "Approve Registration" to grant access or "Reject Registration" if the guidelines are not met.

5. Non-Functional Requirements

This section defines the quality attributes, constraints, and system characteristics required for the effective operation of the Event Bridge AI platform. These requirements ensure system performance, security, scalability, and overall usability.

5.1 Performance Requirements

- **Response Time:** The system shall respond to user actions (e.g., page loads, API requests) within 2–3 seconds under normal load conditions.
- **Search Performance:** Semantic search queries shall return results within 3–5 seconds, depending on dataset size and AI processing.
- **Concurrent Users:** The system shall support multiple concurrent users without significant degradation in performance.
- **Scalability Handling:** The system shall maintain acceptable performance levels as the number of users and events increases.
- **AI Processing Efficiency:** Embedding generation and recommendation processing shall be optimized to minimize latency and resource usage.

5.2 Safety Requirements

- **Data Integrity:** The system shall ensure that user and event data is not lost or corrupted during operations.
- **Failure Handling:** The system shall handle unexpected failures gracefully without causing system crashes.
- **Backup Mechanism:** Regular backups shall be maintained to prevent data loss.
- **User Action Safety:** Critical actions (e.g., event deletion) shall include confirmation mechanisms to prevent accidental operations.

5.3 Security Requirements

- **Authentication:** The system shall implement secure authentication mechanisms for users, communities, and admin.
- **Authorization:** Role-based access control shall be enforced to restrict access to sensitive functionalities.
- **Data Protection:** Sensitive user data shall be securely stored and protected against unauthorized access.
- **Secure Communication:** All communication between client and server shall use secure protocols (e.g., HTTPS).

- **API Security:** External API integrations shall be secured using API keys and proper access control mechanisms.

5.4 Software Quality Attributes

- **Usability:** The system shall provide an intuitive and user-friendly interface for both technical and non-technical users.
- **Reliability:** The system shall operate consistently with minimal downtime.
- **Availability:** The system shall be available to users at all times except during scheduled maintenance.
- **Efficiency:** System resources shall be utilized efficiently to ensure optimal performance.
- **Accuracy:** AI-based recommendations and search results shall maintain a high level of relevance and correctness.

5.5 Business Rules

- **Community Approval Rule:** A community must be approved by the admin before accessing platform features.
- **Event Ownership Rule:** Events must be associated with at least one community.
- **Collaboration Rule:** Collaboration between communities requires mutual agreement.
- **User Registration Rule:** Users must register before participating in events.
- **Feedback Rule:** Only registered participants can provide feedback on events.

5.6 Interoperability

- **API Integration:** The system shall support integration with external AI services such as OpenAI.
- **Cross-System Communication:** The system shall enable communication between the Laravel backend and Python-based AI services.
- **Data Exchange Format:** Standard data formats (e.g., JSON) shall be used for communication between system components.

5.7 Extensibility

- **Modular Architecture:** The system shall be designed using a modular structure to allow the addition of new features.
- **AI Expansion:** The AI module shall support integration of additional models and algorithms.

- **Feature Enhancement:** New functionalities (e.g., analytics dashboards, advanced recommendations) can be added without major restructuring.

5.8 Maintainability

- **Code Structure:** The system shall follow clean coding practices and a standardized structure.
- **Documentation:** Proper technical documentation shall be maintained for all modules.
- **Error Handling:** Clear error messages and logging mechanisms shall be implemented.
- **Update Capability:** System updates and bug fixes shall be applied without major downtime.

5.9 Portability

- **Platform Independence:** The system shall be accessible across different operating systems via web browsers.
- **Deployment Flexibility:** The system shall be deployable on various cloud or VPS environments.

5.10 Reusability

- **Reusable Components:** Common modules (e.g., authentication, notification system) shall be reusable across the system.
- **Service Reusability:** AI services and APIs shall be designed for reuse in other applications or modules.

5.11 Installation

- **Server Setup:** The system shall support installation on Linux-based servers.
- **Dependency Management:** Required frameworks, libraries, and services shall be clearly defined and installable.
- **Configuration:** Environment configuration (e.g., database, API keys) shall be manageable through environment files.
- **Deployment Process:** The system shall support a structured deployment process for smooth setup and updates.

6. Other Requirements

This section specifies additional requirements that are not covered in previous sections but are essential for the complete functioning, usability, and compliance of the Event Bridge AI system.

6.1 On-line User Documentation and Help System Requirements

- **User Help Interface:** The system shall provide an accessible help section within the platform for user guidance.
- **Guided Onboarding:** The system shall include onboarding assistance to help new users and communities understand platform features.
- **FAQ Section:** Frequently Asked Questions shall be available to address common user queries.
- **Error Guidance:** User-friendly error messages and suggestions shall be provided to assist users in resolving issues.
- **Feature Instructions:** Key functionalities (e.g., event creation, registration, collaboration) shall include brief usage instructions.

6.2 Purchased Requirements

- **Third-Party AI Services:** The system may utilize external AI services such as OpenAI for embeddings and NLP tasks.
- **Hosting Services:** Deployment may rely on third-party hosting providers (e.g., VPS or cloud platforms).
- **Domain and SSL:** The system may require domain registration and SSL certificates for secure access.
- **Database Tools:** External tools or services may be used for database hosting and management.

6.3 Licensing Requirements

- **Software Licenses:** All frameworks and libraries used (e.g., Laravel, Python frameworks) shall comply with their respective open-source licenses.
- **API Usage Licensing:** Usage of third-party APIs shall adhere to their licensing terms and usage policies.
- **Content Ownership:** Users and communities retain ownership of content they create on the platform.
- **Compliance:** The system shall comply with applicable software licensing regulations.

6.4 Legal, Copyright, and Other Notices

- **Data Privacy:** The system shall respect user privacy and handle data responsibly.
- **Copyright Protection:** Users shall not upload or share content that violates copyright laws.
- **Terms of Use:** The platform shall define clear terms and conditions for all users.
- **Liability Disclaimer:** The platform shall not be held responsible for user-generated content or external interactions.

6.5 Applicable Standards

- **Web Standards:** The system shall follow standard web development practices (HTML, CSS, JavaScript compatibility).
- **Security Standards:** The system shall follow standard web security practices (e.g., HTTPS, secure authentication).
- **Software Engineering Practices:** The system shall follow structured development methodologies and coding standards.
- **Data Handling Standards:** Proper data management and storage practices shall be followed.

6.6 Reports

- **User Reports:** The system shall provide data related to user registrations and participation.
- **Event Reports:** Communities shall have access to event-related data such as registrations and attendance.
- **Feedback Reports:** The system shall store and display user feedback for events.
- **Usage Analytics:** Basic usage insights (e.g., number of users, events, interactions) shall be available.
- **Administrative Reports:** Admin shall have access to system-level summaries for monitoring platform activity.

7. References

This section provides a curated list of all academic, technical, and industry sources consulted during the preparation of this Software Requirements Specification. References include literature on software engineering best practices, semantic AI techniques, and system design methodologies used in the EventBridge AI system.

All sources are selected based on credibility and their contribution to requirement elicitation, architectural planning, or validation.

[1] Sommerville, I., *Software Engineering*, 10th ed., Pearson Education, 2016: Software Engineering, 10th GLOBAL Edition

[2] Pressman, R. S., and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 9th ed., McGraw-Hill,2020: (PDF) Software Engineering: A Practitioner's Approach 9 th Edition

[3] Gorton, I., *Essential Software Architecture*, 2nd ed., Springer, 2011: Essential Software Architecture | Springer Nature Link

[4] Qdrant Documentation. Available: https://qdrant.tech/documentation/

[5] OpenAI API Documentation. Available: https://platform.openai.com/docs

[6] Laravel Framework Documentation. Available: https://laravel.com/docs

[7] Django Framework Documentation. Available: https://docs.djangoproject.com

[8] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*: https://standards.ieee.org/ieee/830/1222/

[9] World Wide Web Consortium (W3C), *Web Content Accessibility Guidelines (WCAG)*. Available: https://www.w3.org/WAI/standards-guidelines/wcag

[10] ISO/IEC 25010:2011, *Systems and Software Engineering — System and Software Quality Models*: https://www.iso.org/standard/35733.html



SOFTWARE DESIGN DESCRIPTION

for

EventBridge AI
Version 1.0

By

Laiba Rafiq (2022F-SE-064)
Ahmer Zubair (2022F-SE-165)
Muhammad Hashir (2022F-SE-289)

Supervisor
Tauseef Mubeen

Bachelor of Science in Software Engineering (2025-2026)

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to define the architectural blueprint for **EventBridge AI**. This platform is designed to be a centralized AI-powered web solution that connects communities with individuals across the city. It details the system's modules, including the core web application and the AI engine, ensuring a clear roadmap for development and deployment.

1.2 Scope

The scope of EventBridge AI covers the development of a web-based ecosystem that serves two primary user bases: **Communities** (organizers) and **Individuals** (attendees/learners).

- **For Communities:** Tools for event posting, management, and inter-community collaboration.
- **For Individuals:** A unified dashboard for event discovery, profile management, and AI-driven recommendations based on interests.
- **Core Technology:** Integration of a PHP/Laravel frontend with a Python/Django AI backend.

1.3 Project Background

Currently, Individuals and Communities in Karachi are actively involved in organizing and attending events. However, information is fragmented across multiple disparate platforms like WhatsApp, Instagram, Facebook, and LinkedIn. This fragmentation results in low visibility for events and missed opportunities for learning and networking.

1.4 Motivation

The motivation behind this project is to solve the problem of "scattered information". By creating a centralized hub, the platform aims to enhance visibility for Communities and simplify the search process for Individuals, ensuring they don't miss out on valuable opportunities due to a lack of awareness.

1.5 Project Objective

- To provide a centralized platform for Communities to manage and publish events.
- To implement an AI-powered recommendation system to personalize event feeds for Individuals.
- To facilitate inter-community collaboration and unified notification systems.

2. Design Methodology and Software Process Model

2.1 Design Methodology

The system follows an **Object-Oriented Design (OOD)** methodology. This approach allows for the creation of modular classes (e.g., User, Event, Community) that encapsulate data and behavior, ensuring the system is scalable and easier to maintain given the integration between Laravel and Python services.

2.2 Design Pattern

The project utilizes the **MVC** pattern, which is the standard architectural pattern for the Laravel framework.

- **Model:** Manages the data logic and interactions with the MySQL database (e.g., Event, User models).
- **View:** Handles the presentation layer (UI) displayed to Individuals and Communities.
- **Controller:** Processes user inputs and serves as the intermediary between the Model and View.

2.3 Software Process Models

An **Agile** approach is selected to accommodate the iterative nature of developing AI features alongside core web modules.

- **Iterative Development:** The project is divided into sprints (e.g., Frontend, Backend, AI Integration) as outlined in the project timeline.
- **Adaptability:** Allows for continuous testing and refinement of the AI recommendation engine based on user feedback.

3. System Overview

3.1 Architectural Design

The system employs a **Service-Oriented Architecture** (Client-Server model) with a distinct separation between the core application and the AI service.

- **Client Side:** Web browsers accessing the platform.
- **Server Side:**
    - **Web Server:** Runs the Laravel application for core logic and UI.
    - **AI Server:** A Python/Django-based API that handles Vector DB interactions and LLM processing.
- **Database:** A dual-database approach using **MySQL** for relational data and a **Vector Database** for semantic search embeddings.

3.2 Process Flow (Functional Requirement)

Based on the System Flowchart Diagram:

- **User Entry:** The user lands on the application and selects their type: **Community** or **Individual** (User).
- **Authentication:**
    - **Communities:** Register and await approval from the admin before accessing the dashboard.
    - **Individuals:** Register/login to access their personal dashboard.
- **Dashboard Functions:**
    - **Communities:** Can create events, manage collaborations, and view analytics.
    - **Individuals:** Can explore communities, search for events, and view "Recommended Events" generated by the AI Server.
- **Event Interaction:** When a Community publishes an event, it is indexed by the AI Server. Individuals can then view, follow, or register for these events.

4. Project Architecture

4.1 Web Module

1. **Functions (UI Designing & UX Methodology)**
    - **Tools: Figma** is utilized for designing high-fidelity prototypes and wireframes to ensure a professional user experience.
    - **UX Methodology:** The design focuses on accessibility and clarity, ensuring individuals can easily filter events by category (Tech, Gaming), format (Physical/Online), and mode (Free/Paid).
2. **Architecture (Box and Line Diagram)**
    - **Frontend:** HTML/CSS/JS + Laravel Blade Templates.
    - **Backend API:** Laravel handling Authentication, Event Management, and Notifications.
    - **AI Engine:** Python API handling Recommendations and Smart Search (NLP).

4.2 Database Module

- **Type:** Relational Database (MySQL) + Vector Database.
- **Justification:** MySQL is essential for maintaining structured relationships between **Individuals**, **Communities**, and **Events**. The Vector Database is required to store embeddings for the content-based filtering algorithms.
- **DDL (Data Definition Overview):**
    - **Users Table:** Stores Individual profile data.
    - **Communities Table:** Stores Community profiles and approval status.
    - **Events Table:** Stores event metadata (Title, Description, Date).

4.3 Administration Module

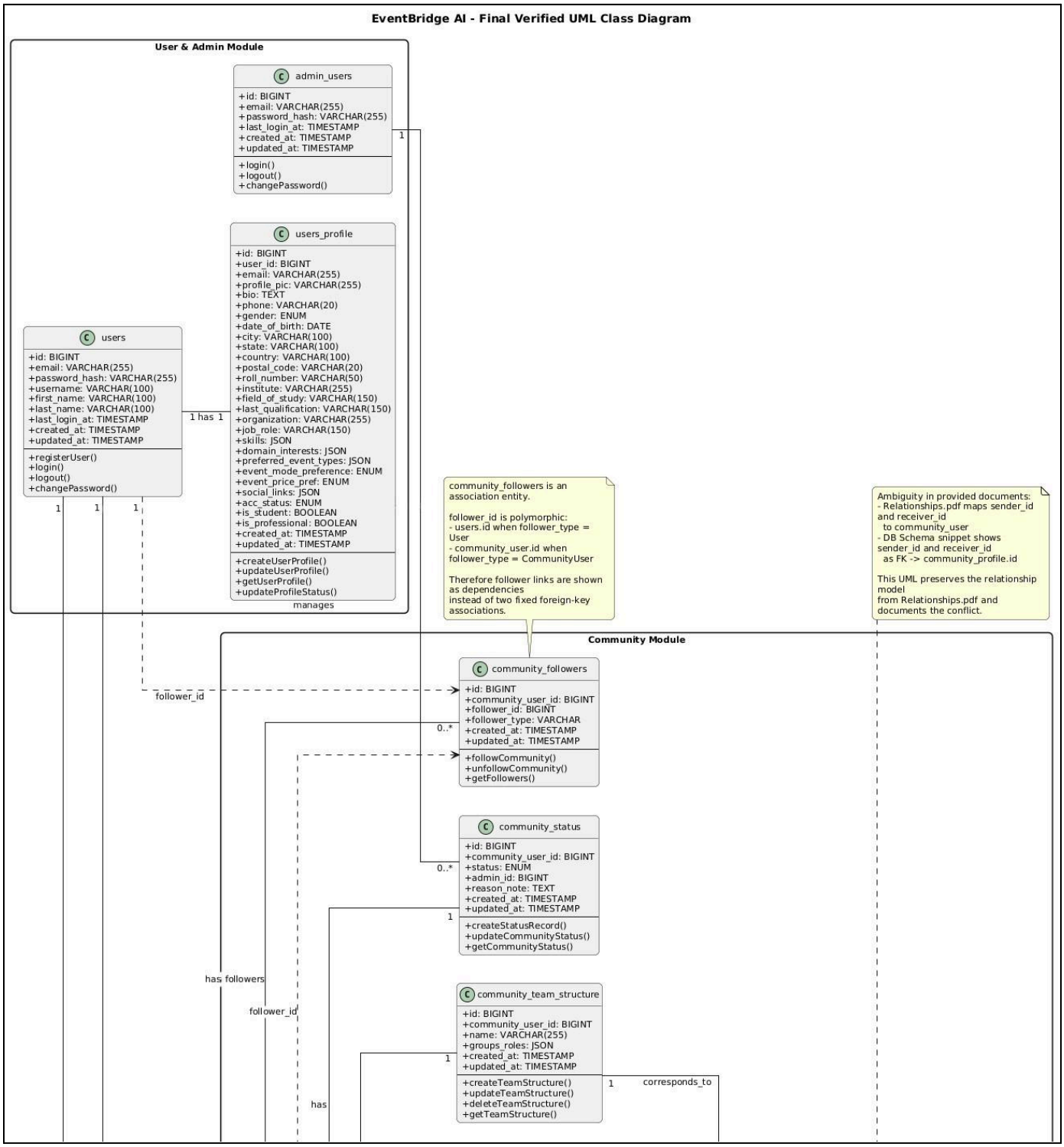
- **Approval System:** Validates Community registration requests.
- **Content Management:** Monitors event postings for compliance.

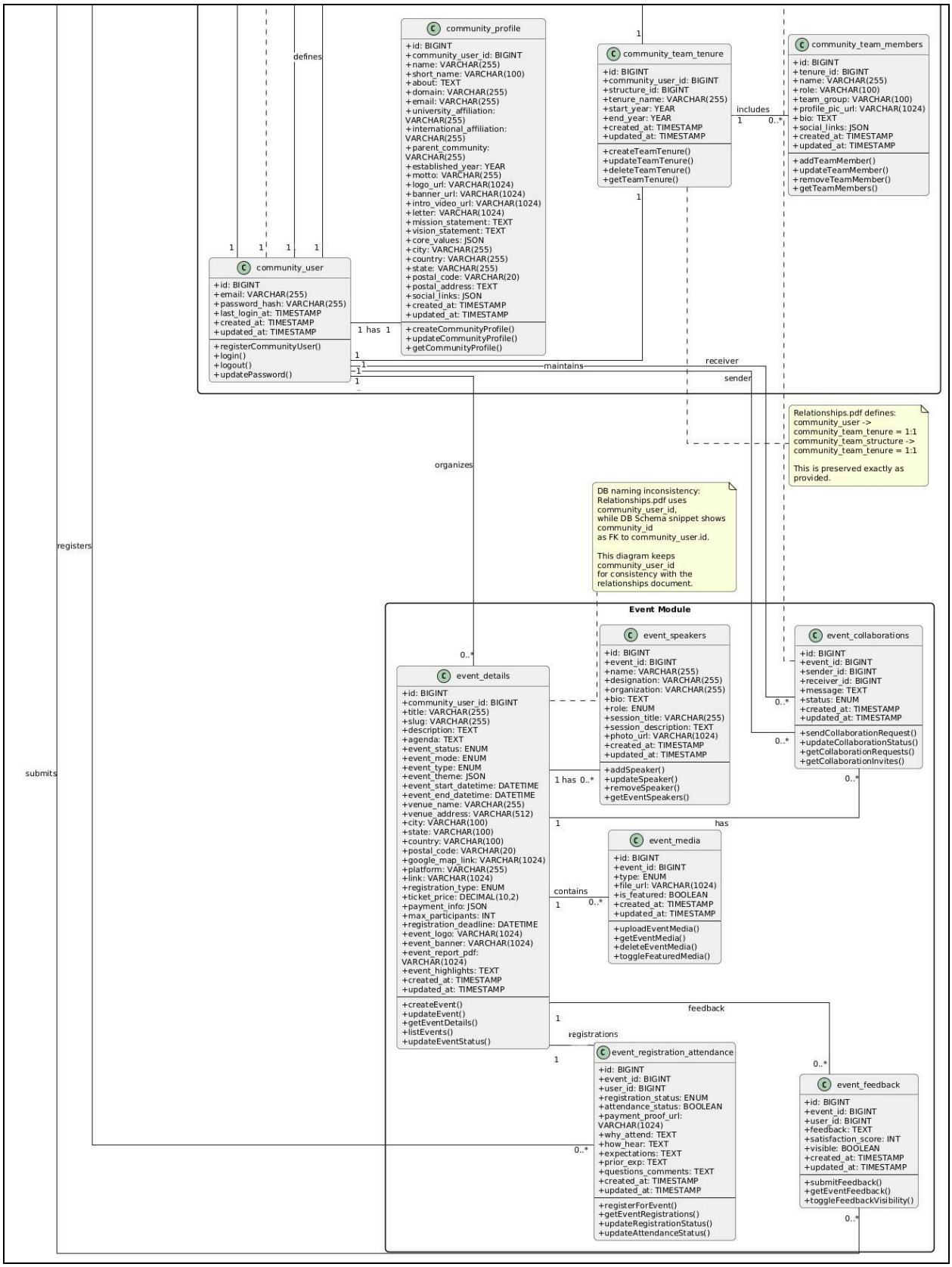
4.4 Data Dictionary

- **Individual:** An end-user who registers to attend events.
- **Community:** An entity (university society or city group) that organizes events.
- **Embedding:** A vector representation of event data used for AI similarity matching.
- **Collaborator:** A secondary community invited to co-host an event.

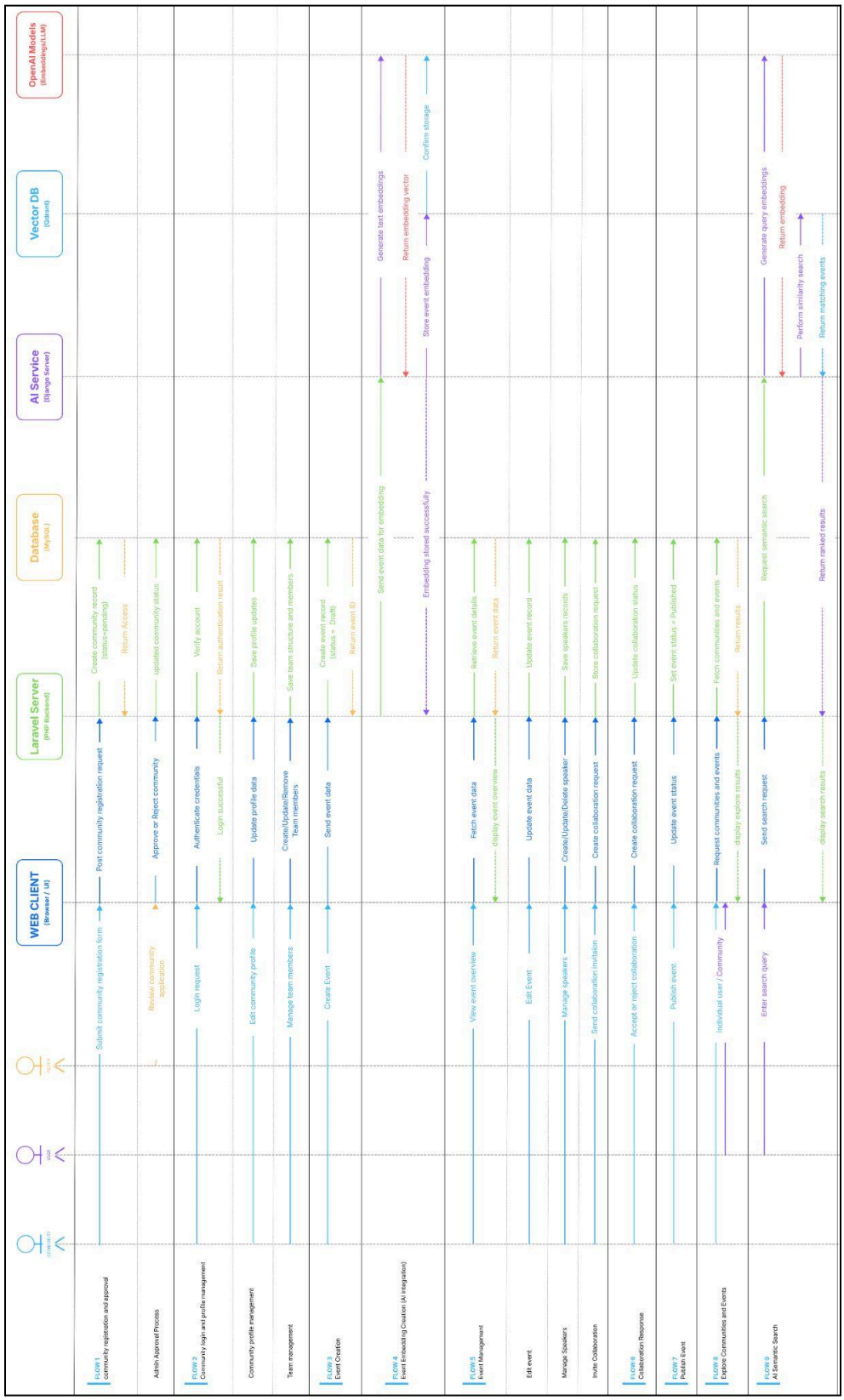
5. System Analysis & Design Overview

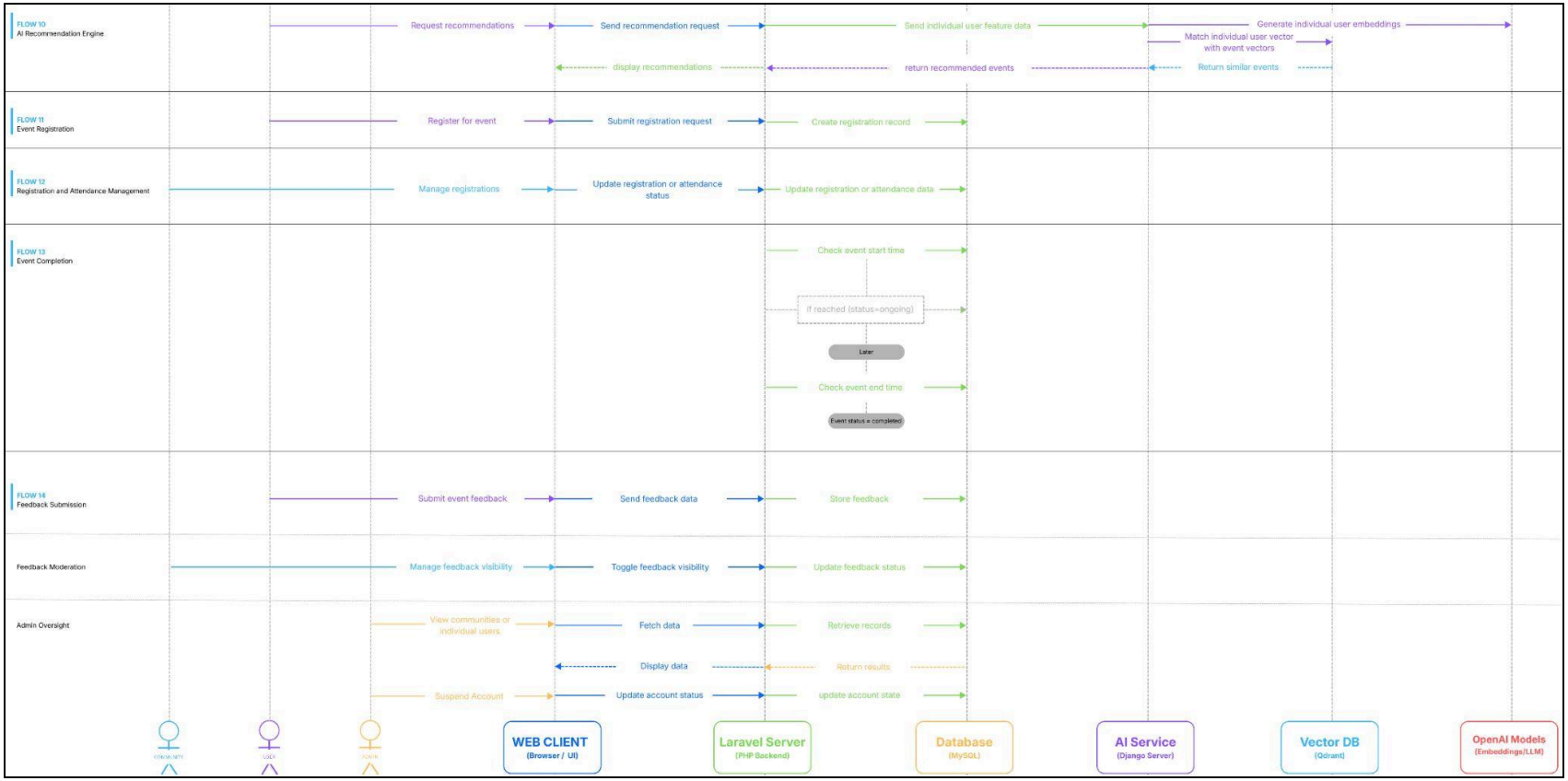
5.1 Class Diagrams



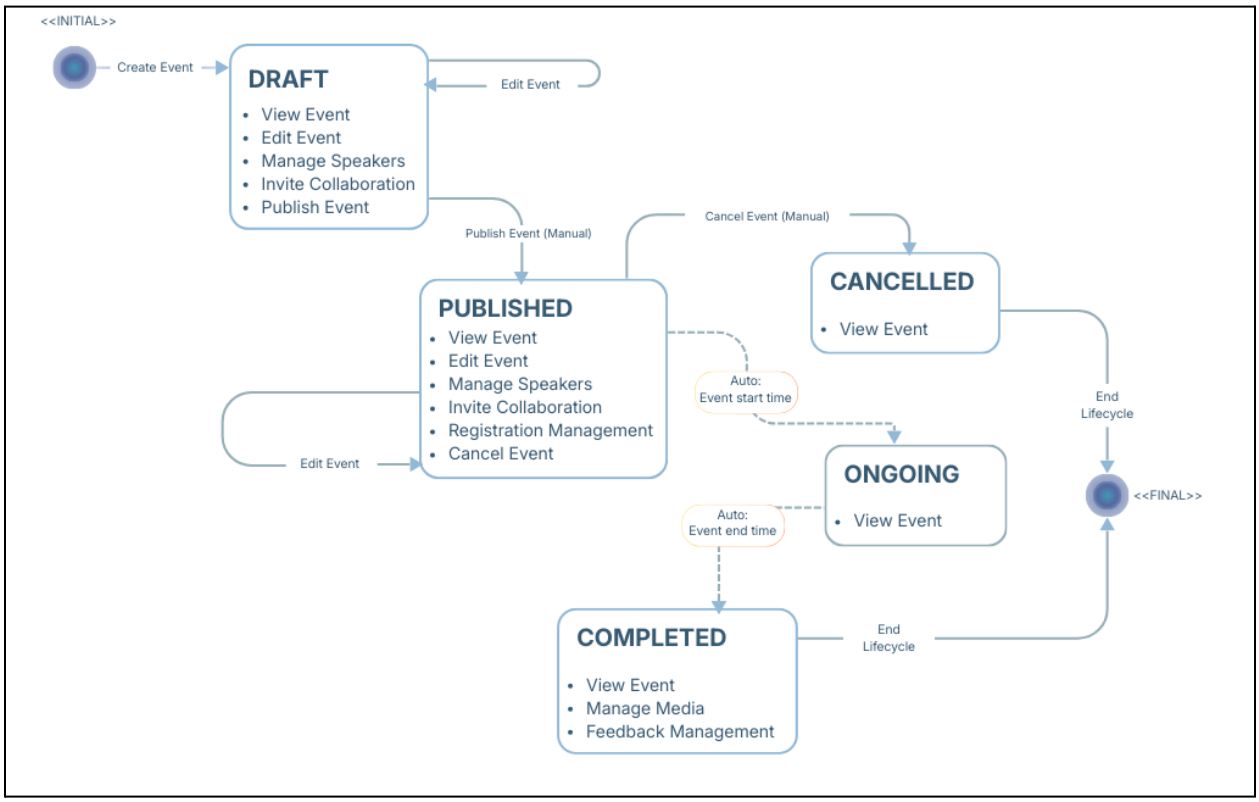

community_profile
+id: BIGINT
+community_user_id: BIGINT
+name: VARCHAR(255)
+short_name: VARCHAR(100)
+about: TEXT
+domain: VARCHAR(255)
+email: VARCHAR(255)
+university_affiliation: VARCHAR(255)
+international_affiliation: VARCHAR(255)
+parent_community: VARCHAR(255)
+established_year: YEAR
+motto: VARCHAR(255)
+logo_url: VARCHAR(1024)
+banner_url: VARCHAR(1024)
+intro_video_url: VARCHAR(1024)
+letter: VARCHAR(1024)
+mission_statement: TEXT
+vision_statement: TEXT
+core_values: JSON
+city: VARCHAR(255)
+country: VARCHAR(255)
+state: VARCHAR(255)
+postal_code: VARCHAR(20)
+postal_address: TEXT
+social_links: JSON
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+createCommunityProfile()
+updateCommunityProfile()
+getCommunityProfile()
community_team_tenure
+id: BIGINT
+community_user_id: BIGINT
+structure_id: BIGINT
+tenure_name: VARCHAR(255)
+start_year: YEAR
+end_year: YEAR
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+createTeamTenure()
+updateTeamTenure()
+deleteTeamTenure()
+getTeamTenure()
community_team_members
+id: BIGINT
+tenure_id: BIGINT
+name: VARCHAR(255)
+role: VARCHAR(100)
+team_group: VARCHAR(100)
+profile_pic_url: VARCHAR(1024)
+bio: TEXT
+social_links: JSON
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+addTeamMember()
+updateTeamMember()
+removeTeamMember()
+getTeamMembers()
community_user
+id: BIGINT
+email: VARCHAR(255)
+password_hash: VARCHAR(255)
+last_login_at: TIMESTAMP
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+registerCommunityUser()
+login()
+logout()
+updatePassword()
defines
includes
1 has 1
maintains
receiver
sender
organizes
registers
submits
Relationships.pdf defines:
community_user ->
community_team_tenure = 1:1
community_team_structure ->
community_team_tenure = 1:1
This is preserved exactly as provided.
DB naming inconsistency:
Relationships.pdf uses
community_user_id,
while DB Schema snippet shows
community_id
as FK to community_user.id.
This diagram keeps
community_user_id
for consistency with the
relationships document.
Event Module
event_details
+id: BIGINT
+community_user_id: BIGINT
+title: VARCHAR(255)
+slug: VARCHAR(255)
+description: TEXT
+agenda: TEXT
+event_status: ENUM
+event_mode: ENUM
+event_type: ENUM
+event_theme: JSON
+event_start_datetime: DATETIME
+event_end_datetime: DATETIME
+venue_name: VARCHAR(255)
+venue_address: VARCHAR(512)
+city: VARCHAR(100)
+state: VARCHAR(100)
+country: VARCHAR(100)
+postal_code: VARCHAR(20)
+google_map_link: VARCHAR(1024)
+platform: VARCHAR(255)
+link: VARCHAR(1024)
+registration_type: ENUM
+ticket_price: DECIMAL(10,2)
+payment_info: JSON
+max_participants: INT
+registration_deadline: DATETIME
+event_logo: VARCHAR(1024)
+event_banner: VARCHAR(1024)
+event_report_pdf: VARCHAR(1024)
+event_highlights: TEXT
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+createEvent()
+updateEvent()
+getEventDetails()
+listEvents()
+updateEventStatus()
event_speakers
+id: BIGINT
+event_id: BIGINT
+name: VARCHAR(255)
+designation: VARCHAR(255)
+organization: VARCHAR(255)
+bio: TEXT
+role: ENUM
+session_title: VARCHAR(255)
+session_description: TEXT
+photo_url: VARCHAR(1024)
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+addSpeaker()
+updateSpeaker()
+removeSpeaker()
+getEventSpeakers()
event_collaborations
+id: BIGINT
+event_id: BIGINT
+sender_id: BIGINT
+receiver_id: BIGINT
+message: TEXT
+status: ENUM
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+sendCollaborationRequest()
+updateCollaborationStatus()
+getCollaborationRequests()
+getCollaborationInvites()
1 has 0..*
has
event_media
+id: BIGINT
+event_id: BIGINT
+type: ENUM
+file_url: VARCHAR(1024)
+is_featured: BOOLEAN
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+uploadEventMedia()
+getEventMedia()
+deleteEventMedia()
+toggleFeaturedMedia()
contains
feedback
registrations
event_registration_attendance
+id: BIGINT
+event_id: BIGINT
+user_id: BIGINT
+registration_status: ENUM
+attendance_status: BOOLEAN
+payment_proof_url: VARCHAR(1024)
+why_attend: TEXT
+how_hear: TEXT
+expectations: TEXT
+prior_exp: TEXT
+questions_comments: TEXT
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+registerForEvent()
+getEventRegistrations()
+updateRegistrationStatus()
+updateAttendanceStatus()
event_feedback
+id: BIGINT
+event_id: BIGINT
+user_id: BIGINT
+feedback: TEXT
+satisfaction_score: INT
+visible: BOOLEAN
+created_at: TIMESTAMP
+updated_at: TIMESTAMP
+submitFeedback()
+getEventFeedback()
+toggleFeedbackVisibility()

5.2 System Sequence Diagrams




FLOW 10
AI Recommendation Engine
Request recommendations
Send recommendation request
Send individual user feature data
Generate individual user embeddings
Match individual user vector with event vectors
display recommendations
return recommended events
Return similar events
FLOW 11
Event Registration
Register for event
Submit registration request
Create registration record
FLOW 12
Registration and Attendance Management
Manage registrations
Update registration or attendance status
Update registration or attendance data
FLOW 13
Event Completion
Check event start time
If reached (status=ongoing)
Later
Check event end time
Event status = completed
FLOW 14
Feedback Submission
Submit event feedback
Send feedback data
Store feedback
Feedback Moderation
Manage feedback visibility
Toggle feedback visibility
Update feedback status
Admin Oversight
View communities or individual users
Fetch data
Retrieve records
Display data
Return results
Suspend Account
Update account status
update account state
WEB CLIENT
(Browser / UI)
Laravel Server
(PHP Backend)
Database
(MySQL)
AI Service
(Django Server)
Vector DB
(Qdrant)
OpenAI Models
(Embeddings/LLM)

5.3 State Transition Diagram



Report On Sustainable Development Goal(s)

for

EventBridge AI

By
Laiba Rafiq (2022F-SE-064)
Ahmer Zubair (2022F-SE-165)
Muhammad Hashir (2022F-SE-289)

Supervisor
Tauseef Mubeen

Bachelor of Science in Software Engineering (2025-2026)

Index

1. Project Title

EventBridge AI: An Intelligent Platform for Centralizing Communities and Empowering Connections.

2. Introduction

In an increasingly digital yet fragmented world, meaningful community engagement and networking often face logistical and communication barriers. EventBridge AI is designed to bridge this gap by providing an intelligent, centralized platform that empowers communities to connect, collaborate, and grow. This report outlines how EventBridge AI aligns with the United Nations Sustainable Development Goals (SDGs), demonstrating the platform's commitment to fostering sustainable innovation, inclusive communities, and economic growth through advanced AI-driven networking.

3. SDG Alignment

3.1 Primary SDG

SDG 9 (Industry, Innovation, and Infrastructure):

EventBridge AI directly contributes to SDG 9 by upgrading technological infrastructure and fostering innovation. By leveraging Artificial Intelligence to optimize event management, match like-minded individuals, and streamline community operations, the platform represents a significant step forward in digital infrastructure for social and professional networking.

3.2 Secondary SDGs

- **SDG 4 (Quality Education):** EventBridge AI facilitates continuous learning and knowledge transfer by connecting students, educators, and industry professionals. By centralizing academic and tech communities (such as university developer groups), the platform provides accessible avenues for skill-sharing, mentorship, and extracurricular education beyond the traditional classroom.
- **SDG 17 (Partnerships for the Goals):** At the core of EventBridge AI is the goal of bringing people and organizations together. It acts as a catalyst for forming cross-sector partnerships, encouraging collaboration between individuals, academic institutions, and businesses to share knowledge, resources, and achieve common goals.

4. Project Impact

4.1 Quantitative Impact

- **User Engagement:** Expected to onboard and engage 500 - 1000+ users within the first year, which includes individuals and communities.
- **Event Efficiency:** Prediction is to support communities in managing 100s of events annually, smoothly improving the complete lifecycle management. The engagement rate to events and communities is to be increased by 30 to 50% by this platform, through personalized AI recommendations.
- **Networking Metrics:** Estimated collaborations to be facilitated are around 50+ per year.

4.2 Qualitative Impact

- **Enhanced Community Cohesion:** Ease for individuals in exploring and discovering events and communities, enhancing and providing more opportunities in learning and enhancing skills beyond traditional education.
- **Empowered Networking:** With the use of AI-powered recommendations, the system ensures visibility and accessibility of events to individuals.
- **Knowledge Transfer:** Intercommunity collaboration will increase partnerships between communities, encouraging shared knowledge and collective growth. The platform streamlines the entire process of event creation and management, reducing the manual efforts required to manage events efficiently.

5. Future Implications

5.1 Potential for Scalability

The architecture of EventBridge AI is inherently scalable. While it can successfully launch within a specific ecosystem (such as a university campus or a specific tech community), its cloud-based and AI-driven nature allows for seamless expansion. It can easily scale to accommodate multi-city corporate events, global academic conferences, or nationwide professional syndicates without a drop in performance.

5.2 Policy Recommendations

To maximize the positive impact of platforms like EventBridge AI, the following policy considerations are recommended:

- **Data Privacy & Ethics:** Institutions utilizing the platform must enforce strict data protection policies to ensure that user data used by the AI for matching is handled securely and transparently.
- **Digital Inclusivity:** Organizations should provide necessary digital literacy training and internet access to ensure all community members can benefit from the platform, preventing a digital divide.

5.3 Long-term Impact

In the long run, EventBridge AI has the potential to transform how professional and social ecosystems operate. It will shift the paradigm from reactive event attendance to proactive, continuous community building. This sustained engagement will foster lifelong learning, resilient professional networks, and a culture of continuous innovation.

6. Conclusion

EventBridge AI is more than just an event management tool; it is a catalyst for sustainable community development. By aligning strongly with SDGs 9, 4, and 17, the platform proves its viability not only as a technological innovation but as a socially responsible venture. Through its intelligent matching and centralized hubs, EventBridge AI is poised to leave a lasting, positive impact on how society connects and collaborates.

7. References

[1] United Nations. (2015). Transforming our world: the 2030 Agenda for Sustainable Development: https://sdgs.un.org/2030agenda